Lab Guide: Deploying FastAPI in a Docker Container with Nginx

Objective:

This lab will guide you through setting up a FastAPI application inside a Docker container and exposing it using Nginx.

Prerequisites:

- A Linux-based server (Ubuntu recommended)
- Docker and Docker Compose installed
- Basic understanding of Linux commands

Step 1: Install Docker

---

sudo apt update

sudo apt install -y docker.io

sudo systemctl enable --now docker

Verify installation:

docker --version

Step 2: Create a FastAPI App

---

1. Create a Project Directory:

mkdir ~/docker-app && cd ~/docker-app

2. Create app.py:

echo "from fastapi import FastAPI

app = FastAPI()

@app.get('/')

def read_root():

    return {"message": "Hello, FastAPI!"}" > app.py

Step 3: Create a Dockerfile

---

echo "FROM tiangolo/uvicorn-gunicorn-fastapi:python3.9

WORKDIR /app
COPY app.py /app/
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "3000"]" > Dockerfile

Build and run the container:
docker build -t fastapi-app .
docker run -d --name my-fastapi-app -p 3000:3000 fastapi-app

Check running containers:
docker ps

Step 4: Install and Configure Nginx
---
sudo apt install -y nginx

Edit Nginx configuration:
sudo nano /etc/nginx/sites-available/default

Replace content with:
```
server {
    listen 80;
    server_name your_domain_or_ip;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Restart Nginx:
sudo systemctl restart nginx

Step 5: Test the Setup

Check if FastAPI is running:
curl http://localhost:3000/

If everything is working, you should see:
{"message": "Hello, FastAPI!"}

Now, open your browser and visit:
http://your_server_ip/

Troubleshooting Tips:

- If the container is not running, check logs:
docker logs my-fastapi-app

- If Nginx is not responding, check its status:
sudo systemctl status nginx

- Make sure the firewall allows traffic on port 80:
sudo ufw allow 80/tcp

End of Lab